Linux-RT 应用开发案例

Revision History

Draft Date	Revision No.	Description
2024/10/22	V1.1	1. 更新“Linux、Linux-RT 实时性对比”小节。 2. 更新 rt_gpio_ctr、rt_input 案例的“案例测试”小节。
2024/06/25	V1.0	1. 初始版本。

目　　录

前　言 3
1 Linux 系统实时性测试 4
1.1　Linux、Linux-RT 实时性对比 4
1.2　Linux-RT 性能测试 8
1.2.1　CPU 空载状态 8
1.2.2　CPU 满负荷状态 9
1.2.3　隔离 CPU 核心状态 10
1.2.4　统计结果分析 11
2 rt_gpio_ctrl 案例 15
2.1　案例说明 15
2.2　案例测试 16
2.3　案例编译 18
2.4　关键代码说明 18
3 rt_input 案例 20
3.1　案例说明 21
3.2　案例测试 21
3.3　案例编译 23
3.4　关键代码说明 24
更多帮助 26

前　言

本文档主要通过 Cyclictest 延迟检测工具，以及基于 Linux-RT 的应用程序评估 Linux-RT 系统的实时性，适用开发环境如下：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

Linux 开发环境：VMware15.5.5、Ubuntu20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71、Linux-RT-5.15.71

LinuxSDK：Real-Time Edge Software 2.5

我司默认使用的是 Linux 内核，同时提供了 Linux-RT 内核位于产品资料“4-软件资料\Linux\Kernel\image\linux-5.15.71-rt51-[版本号]-[Git 系列号]”目录下，版本号、Git 序列号以实际情况为准。请按照如下方法替换为 Linux-RT 内核。

我司提供的制卡工具包已包含 Linux 内核镜像、Linux 内核模块，以及 Linux-RT 内核镜像、Linux-RT 内核模块。默认情况下，直接使用制卡工具包制作的是 Linux 内核版本的 Linux 系统启动卡。如需制作 Linux-RT 内核版本的 Linux 系统启动卡，请按照如下方法替换内核镜像和内核模块，然后参考《Linux 系统启动卡制作及固化》文档的“Linux 系统启动卡制作”章节，进行系统启动卡制作。

Host# cp ./boot/Image ./boot/Image-linux　　//备份 Linux 内核镜像

Host# cp ./boot/Image-rt ./boot/Image　　//替换为 Linux-RT 内核镜像

Host# mv ./filesystem/modules ./filesystem/modules-linux　　//备份 Linux 内核模块

Host# cp -r ./filesystem/modules-rt ./filesystem/modules　　//替换为 Linux-RT 内核模块

```
tronlong@tronlong-virtual-machine:~/Imx8mp/mksdboot_10_10_10$ cp ./boot/Image ./boot/Image-linux
tronlong@tronlong-virtual-machine:~/Imx8mp/mksdboot_10_10_10$ cp ./boot/Image-rt ./boot/Image
tronlong@tronlong-virtual-machine:~/Imx8mp/mksdboot_10_10_10$ mv ./filesystem/modules ./filesystem/modules-linux
tronlong@tronlong-virtual-machine:~/Imx8mp/mksdboot_10_10_10$ cp -r ./filesystem/modules-rt ./filesystem/modules
tronlong@tronlong-virtual-machine:~/Imx8mp/mksdboot_10_10_10$
```

图 1

1 Linux 系统实时性测试

本章节主要介绍使用 Cyclictest 延迟检测工具测试 Linux 系统实时性的方法。Cyclictest 是 rt-tests 测试套件下的一个测试工具，也是 rt-tests 下使用最广泛的测试工具，一般主要用来测试内核的延迟，从而判断内核的实时性。

Cyclictest 常用于实时系统的基准测试，是评估实时系统相对性能的最常用工具之一。Cyclictest 反复测量并精确统计线程的实际唤醒时间，以提供有关系统的延迟信息。它可测量由硬件、固件和操作系统引起的实时系统的延迟。

为了测量延迟，Cyclictest 运行一个非实时主线程（调度类 SCHED_OTHER），该线程以定义的实时优先级（调度类 SCHED_FIFO）启动定义数量的测量线程。测量线程周期性地被一个到期的计时器（循环报警）所定义的间隔唤醒，随后计算有效唤醒时间，并通过共享内存将其传递给主线程。主线程统计延迟值并打印最小、最大和平均延迟时间。

参考链接：https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/cyclictest/start?s[]=cyclictest。

1.1 Linux、Linux-RT 实时性对比

本次测试结合 Iperf 和 Cyclictest 工具，对比测试基于 Linux-RT-5.15.71 内核和 Linux-5.15.71 内核的系统实时性能。此处使用 Iperf 工具不断触发系统中断，提高中断处理负载，以便更好测试系统实时特性。

在 Ubuntu 执行如下命令查看 IP 地址，并以服务器模式启动 Iperf 测试。

```
Host#    ifconfig
Host#    iperf3 -s
```

```
tronlong@tronlong-virtual-machine:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:65:77:01:e5  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.13.43  netmask 255.255.255.0  broadcast 192.168.13.255
        inet6 fe80::2269:1000:a526:4ca5  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:0f:95:fa  txqueuelen 1000  (Ethernet)
        RX packets 838  bytes 487543 (487.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 301  bytes 30651 (30.6 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 159  bytes 15421 (15.4 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 159  bytes 15421 (15.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

tronlong@tronlong-virtual-machine:~$ iperf3 -s
-----------------------------------------------------------
Server listening on 5201
-----------------------------------------------------------
```

图 2

分别使用 Linux-RT-5.15.71 内核和 Linux-5.15.71 内核启动评估板，执行如下命令以客户端模式启动 Iperf，并连接至服务器端（Ubuntu 系统）。"192.168.13.43"为 Ubuntu 的 IP 地址，"-t3600"设置测试时间为 3600 秒，"&"表示让程序在后台运行。

Target# iperf3 -c 192.168.13.43 -d -t3600 >/dev/null 2>& 1 &

```
root@IMX8MP-Tronlong:~# iperf3 -c 192.168.13.43 -d -t3600 >/dev/null 2>& 1 &
[1] 917
root@IMX8MP-Tronlong:~#
```

图 3

请将产品资料“4-软件资料\Demo\rt-linux-demos\Cyclictest\bin\”目录下的 cyclictest 文件拷贝至评估板文件系统，执行如下命令使用 Cyclictest 工具测试系统实时性。

Target# cyclictest -t5 -p98 -m -D10m

```
root@IMX8MP-Tronlong:~# cyclictest -t5 -p98 -m -D10m
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 0.49 0.57 0.46 1/248 934

T: 0 (  919) P:98 I:1000 C: 599704 Min:      6 Act:   10 Avg:   11 Max:      97
T: 1 (  920) P:98 I:1500 C: 399801 Min:      6 Act:    8 Avg:   10 Max:      75
T: 2 (  921) P:98 I:2000 C: 299850 Min:      6 Act:   16 Avg:   10 Max:      70
T: 3 (  922) P:98 I:2500 C: 239879 Min:      6 Act:    9 Avg:   11 Max:      70
T: 4 (  923) P:98 I:3000 C: 199899 Min:      7 Act:   16 Avg:   13 Max:      88
root@IMX8MP-Tronlong:~# uname -a
Linux IMX8MP-Tronlong 5.15.71-rt51-g0785f0a61619 #1 SMP PREEMPT_RT Fri Sep 20 14:12:45 CST 2024 aarch64 aarch64 aarch64 GNU/Li
nux
root@IMX8MP-Tronlong:~#
```

图 4 Linux-RT-5.15.71 内核测试结果

```
root@IMX8MP-Tronlong:~# uname -a
Linux IMX8MP-Tronlong 5.15.71-rt51-g0785f0a61619 #1 SMP Fri Sep 20 11:31:38 CST 2024 aarch64 aarch64 aarch64 GNU/L
inux
root@IMX8MP-Tronlong:~# cyclictest -t5 -p98 -m -D10m
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 0.10 0.11 0.12 1/196 871

T: 0 (  858) P:98 I:1000 C: 599733 Min:      5 Act:   21 Avg:   24 Max:     450
T: 1 (  859) P:98 I:1500 C: 399821 Min:      4 Act:   10 Avg:    7 Max:     143
T: 2 (  860) P:98 I:2000 C: 299999 Min:      4 Act:    9 Avg:    8 Max:     144
T: 3 (  861) P:98 I:2500 C: 239999 Min:      5 Act:   10 Avg:    8 Max:     125
T: 4 (  862) P:98 I:3000 C: 199999 Min:      6 Act:   11 Avg:   27 Max:     469
root@IMX8MP-Tronlong:~#
```

图 5 Linux-5.15.71 内核测试结果

表 1 Cyclictest 测试结果解析

参数	解析
T(Thread)	线程索引和线程 ID
P(Priority)	线程的优先级
I(Interval)	延迟，测量线程的预期唤醒周期
C(Count)	测量延迟的次数
Min(Minimum)	测量的最小延迟（单位 us）
Act(Actual)	最近一次测量的延迟（单位 us）
Avg(Average)	平均延迟（单位 us）
Max(Maximum)	测量的最大延迟（单位 us）

对比测试数据，可看到基于 Linux-RT-5.15.71 内核的系统的延迟更加稳定，平均延迟、最大延迟更低，系统实时性更佳。

Cyclictest 命令参数解析可执行"cyclictest --help"查看，如下图所示。

```
root@IMX8MP-Tronlong:~# cyclictest --help
cyclictest V 2.30
Usage:
cyclictest <options>

-a [CPUSET] --affinity     Run thread #N on processor #N, if possible, or if CPUSET
                           given, pin threads to that set of processors in round-
                           robin order.  E.g. -a 2 pins all threads to CPU 2,
                           but -a 3-5,0 -t 5 will run the first and fifth
                           threads on CPU (0),thread #2 on CPU 3, thread #3
                           on CPU 4, and thread #5 on CPU 5.
-A USEC  --aligned=USEC    align thread wakeups to a specific offset
-b USEC  --breaktrace=USEC send break trace command when latency > USEC
-c CLOCK --clock=CLOCK     select clock
                           0 = CLOCK_MONOTONIC (default)
                           1 = CLOCK_REALTIME
         --default-system  Don't attempt to tune the system from cyclictest.
                           Power management is not suppressed.
                           This might give poorer results, but will allow you
                           to discover if you need to tune the system
-d DIST  --distance=DIST   distance of thread intervals in us, default=500
-D       --duration=TIME   specify a length for the test run.
                           Append 'm', 'h', or 'd' to specify minutes, hours or days.
-F       --fifo=<path>     create a named pipe at path and write stats to it
-h       --histogram=US    dump a latency histogram to stdout after the run
                           US is the max latency time to be tracked in microseconds
                           This option runs all threads at the same priority.
-H       --histofall=US    same as -h except with an additional summary column
         --histfile=<path> dump the latency histogram to <path> instead of stdout
-i INTV  --interval=INTV   base interval of thread in us default=1000
         --json=FILENAME   write final results into FILENAME, JSON formatted
         --laptop          Save battery when running cyclictest
                           This will give you poorer realtime results
                           but will not drain your battery so quickly
         --latency=PM_QOS  power management latency target value
                           This value is written to /dev/cpu_dma_latency
                           and affects c-states. The default is 0
-l LOOPS --loops=LOOPS     number of loops: default=0(endless)
         --mainaffinity=CPUSET
                           Run the main thread on CPU #N. This only affects
                           the main thread and not the measurement threads
-m       --mlockall        lock current and future memory allocations
-M       --refresh_on_max  delay updating the screen until a new max
                           latency is hit. Useful for low bandwidth.
-N       --nsecs           print results in ns instead of us (default us)
-o RED   --oscope=RED      oscilloscope mode, reduce verbose output by RED
-p PRIO  --priority=PRIO   priority of highest prio thread
         --policy=NAME     policy of measurement thread, where NAME may be one
                           of: other, normal, batch, idle, fifo or rr.
         --priospread      spread priority levels starting at specified value
-q       --quiet           print a summary only on exit
-r       --relative        use relative timer instead of absolute
-R       --resolution      check clock resolution, calling clock_gettime() many
                           times.  List of clock_gettime() values will be
                           reported with -X
         --secaligned [USEC] align thread wakeups to the next full second
                           and apply the optional offset
-s       --system          use sys_nanosleep and sys_setitimer
-S       --smp             Standard SMP testing: options -a -t and same priority
                           of all threads
        --spike=<trigger>  record all spikes > trigger
        --spike-nodes=[num of nodes]
                           These are the maximum number of spikes we can record.
                           The default is 1024 if not specified
-t       --threads         one thread per available processor
-t [NUM] --threads=NUM     number of threads:
```

图 6

```
-t [NUM] --threads=NUM     number of threads:
                           without NUM, threads = max_cpus
                           without -t default = 1
         --tracemark       write a trace mark when -b latency is exceeded
-u       --unbuffered      force unbuffered output for live processing
-v       --verbose         output values on stdout for statistics
                           format: n:c:v n=tasknum c=count v=value in us
         --dbg_cyclictest  print info useful for debugging cyclictest
-x       --posix_timers    use POSIX timers instead of clock_nanosleep.
root@IMX8MP-Tronlong:~#
```

图 7

1.2 Linux-RT 性能测试

本次测试分别在 CPU 空载、满负荷（运行 stress 压力测试工具）、隔离 CPU 核心的情况下，对比评估 Linux-RT 内核的系统实时性。

1.2.1 CPU 空载状态

评估板上电启动，进入评估板文件系统执行如下命令，修改内核 printk 日志等级，避免内核打印信息影响实时测试。

Target# echo 1 > /proc/sys/kernel/printk

调整内存分配策略为“2”，禁用内存过度使用。避免出现 OOM(Out-of-Memory) Killer 攻击某些进程而产生延迟，影响测试结果。

Target# echo 2 > /proc/sys/vm/overcommit_memory

```
root@IMX8MP-Tronlong:~# echo 1 > /proc/sys/kernel/printk
root@IMX8MP-Tronlong:~# echo 2 > /proc/sys/vm/overcommit_memory
root@IMX8MP-Tronlong:~#
```

图 8

执行如下命令，基于 CPU 空载状况下测试系统的实时性。测试指令需运行 12 小时，请保持评估板长时间稳定工作，测试完成后将生成统计结果 no_load_output 文件。

Target# cyclictest -m -Sp99 -i1000 -h800 -D12h -q > no_load_output

```
root@IMX8MP-Tronlong:~# cyclictest -m -Sp99 -i1000 -h800 -D12h -q > no_load_output
root@IMX8MP-Tronlong:~#
```

图 9

```
root@IMX8MP-Tronlong:~# ls
cyclictest  no_load_output
root@IMX8MP-Tronlong:~#
```

图 10

参数解析：

-m：锁定当前和将来的内存分配；

-S：采用标准 SMP 测试；

-p：设置线程优先级；

-i：设置线程的基本间隔；

-h：运行后将延迟直方图转储至标准输出，亦可指定要跟踪的最大延时时间（以微秒为单位）；

-D：指定测试运行时长，附加 m（分钟）、h（小时）、d（天）指定；

-q：运行时不打印相关信息；

1.2.2 CPU 满负荷状态

评估板上电启动，进入评估板文件系统执行如下命令，修改内核 printk 日志等级，避免内核打印信息影响实时测试。

Target# echo 1 > /proc/sys/kernel/printk

调整内存分配策略为“2”，禁用内存过度使用。避免出现 OOM(Out-of-Memory) Killer 攻击某些进程而产生延迟，影响测试结果。

Target# echo 2 > /proc/sys/vm/overcommit_memory

```
root@IMX8MP-Tronlong:~# echo 1 > /proc/sys/kernel/printk
root@IMX8MP-Tronlong:~# echo 2 > /proc/sys/vm/overcommit_memory
```

图 11

执行如下命令，运行 stress 压力测试工具，使得 CPU 处于满负荷状态。再使用 cyclictest 工具测试 CPU 满负荷状态下的系统实时性能。测试指令需运行 12 小时，请保持评估板长时间稳定工作，测试完成后将生成统计结果 overload_output 文件。

Target# stress --cpu 4 --io 4 --vm 4 --vm-bytes 64M --timeout 43200s &

Target# cyclictest -m -Sp99 -i1000 -h800 -D12h -q > overload_output

```
root@IMX8MP-Tronlong:~# stress --cpu 4 --io 4 --vm 4 --vm-bytes 64M --timeout 43200s &
[1] 834
root@IMX8MP-Tronlong:~# stress-ng: info:  [834] setting to a 43200 second (12 hours, 0.00 secs) ru
n per stressor
stress-ng: info:  [834] dispatching hogs: 4 cpu, 4 io, 4 vm

root@IMX8MP-Tronlong:~# cyclictest -m -Sp99 -i1000 -h800 -D12h -q > overload_output
stress-ng: info:  [834] successful run completed in 43200.46s (12 hours, 0.46 secs)
[1]+  Done                    stress --cpu 4 --io 4 --vm 4 --vm-bytes 64M --timeout 43200s
root@IMX8MP-Tronlong:~#
```

图 12

```
root@IMX8MP-Tronlong:~# ls
cyclictest  overload_output
root@IMX8MP-Tronlong:~#
```

图 13

参数解析：

--cpu：指定压力测试的进程个数；

--io：指定 I/O 测试的进程个数；

--vm：指定内存测试的进程个数；

--vm-bytes：指定每个内存测试进程中分配内存的大小；

--timeout：指定测试时长；

1.2.3 隔离 CPU 核心状态

本次测试以隔离 CPU3 核心为例，通过降低系统上所运行的其他进程对隔离 CPU3 产生的延迟影响，确保 CPU3 进程的正常运行，进而评估 Linux-RT 内核的系统实时性。

评估板上电启动后，在 U-Boot 倒计时结束之前按下空格键进入 U-Boot 命令行模式，执行如下命令，修改环境变量，隔离 CPU3 核心。

U-Boot# setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} isolcpus=3"

U-Boot# saveenv

U-Boot# reset

```
u-boot=> setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} isolcpus=3"
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 14

进入评估板文件系统，执行如下命令，修改内核 printk 日志等级，避免内核打印信息影响实时测试。

Target# echo 1 > /proc/sys/kernel/printk

调整内存分配策略为“2”，禁用内存过度使用。避免出现 OOM(Out-of-Memory) Killer 攻击某些进程而产生延迟，影响测试结果。

Target# echo 2 > /proc/sys/vm/overcommit_memory

```
root@IMX8MP-Tronlong:~# echo 1 > /proc/sys/kernel/printk
root@IMX8MP-Tronlong:~# echo 2 > /proc/sys/vm/overcommit_memory
root@IMX8MP-Tronlong:~#
```

图 15

执行如下命令，运行 stress 压力测试工具，使得 CPU 处于满负荷状态。再使用 taskset 工具将 cyclictest 测试程序运行在 CPU3 核心，测试 CPU3 核心满负荷状态下的系统实时性能。测试指令需运行 12 小时，请保持评估板长时间稳定工作，测试完成后将生成统计结果 iso_overload_output 文件。

Target# stress --cpu 4 --io 4 --vm 4 --vm-bytes 64M --timeout 43200s &

Target# taskset -c 0-3 cyclictest -m -Sp99 -i1000 -h800 -D12h -q > iso_overload_output

```
root@IMX8MP-Tronlong:~# stress --cpu 4 --io 4 --vm 4 --vm-bytes 64M --timeout 43200s &
[1] 886
root@IMX8MP-Tronlong:~# stress-ng: info:  [886] setting to a 43200 second (12 hours, 0.00 secs) run per stressor
stress-ng: info:  [886] dispatching hogs: 4 cpu, 4 io, 4 vm

root@IMX8MP-Tronlong:~# taskset -c 0-3 cyclictest -m -Sp99 -i1000 -h800 -D12h -q > iso_overload_output
stress-ng: info:  [886] successful run completed in 43200.40s (12 hours, 0.40 secs)
[1]+  Done                    stress --cpu 4 --io 4 --vm 4 --vm-bytes 64M --timeout 43200s
root@IMX8MP-Tronlong:~#
```

图 16

```
root@IMX8MP-Tronlong:~# ls
cyclictest  iso_overload_output
root@IMX8MP-Tronlong:~#
```

图 17

1.2.4 统计结果分析

我司已提供脚本文件 get_histogram.sh 用于绘制统计结果直方图，位于产品资料“4-软件资料\Demo\rt-linux-demos\Cyclictest\bin\”目录下，请将该脚本文件拷贝至 Ubuntu 工作目录下。

```
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$ ls
get_histogram.sh
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$
```

图 18

在 Ubuntu 系统执行如下命令，安装 gnuplot 工具。

Host# sudo apt-get install gnuplot

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get install gnuplot
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  aglfn gnuplot-data gnuplot-qt libwxbase3.0-0v5 libwxgtk3.0-gtk3-0v5
Suggested packages:
  gnuplot-doc
The following NEW packages will be installed:
  aglfn gnuplot gnuplot-data gnuplot-qt libwxbase3.0-0v5 libwxgtk3.0-gtk3-0v5
0 upgraded, 6 newly installed, 0 to remove and 60 not upgraded.
Need to get 6,466 kB of archives.
After this operation, 24.9 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu focal/universe amd64 aglfn all 1.7+g
it20191031.4036a9c-2 [30.6 kB]
Get:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu focal/universe amd64 gnuplot-data al
l 5.2.8+dfsg1-2 [56.5 kB]
Get:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu focal/universe amd64 libwxbase3.0-0v
5 amd64 3.0.4+dfsg-15build1 [982 kB]
Get:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu focal/universe amd64 libwxgtk3.0-gtk
3-0v5 amd64 3.0.4+dfsg-15build1 [4,359 kB]
Get:5 http://mirrors.tuna.tsinghua.edu.cn/ubuntu focal/universe amd64 gnuplot-qt amd6
4 5.2.8+dfsg1-2 [1,034 kB]
Get:6 http://mirrors.tuna.tsinghua.edu.cn/ubuntu focal/universe amd64 gnuplot all 5.2
.8+dfsg1-2 [3,792 B]
```

图 19

（1） CPU 空载状态

请将 CPU 空载状态下的统计结果 no_load_output 文件拷贝至 Windows 工作目录，使用 Windows 文本工具打开该文件并拖动至文件末尾，可查看 Linux 系统每个核心(CPU0~CPU3)的最小延迟(Min Latencies)、平均延迟(Avg Latencies)、最大延迟(Max Latencies)统计结果。

```
000790 000000      000000    000000    000000
000791 000000      000000    000000    000000
000792 000000      000000    000000    000000
000793 000000      000000    000000    000000
000794 000000      000000    000000    000000
000795 000000      000000    000000    000000
000796 000000      000000    000000    000000
000797 000000      000000    000000    000000
000798 000000      000000    000000    000000
000799 000000      000000    000000    000000
# Total: 043200000 043199986 043199971 043199956
# Min Latencies: 00006 00006 00006 00006
# Avg Latencies: 00008 00008 00009 00008
# Max Latencies: 00040 00042 00043 00052
# Histogram Overflows: 00000 00000 00000 00000
# Histogram Overflow at cycle number:
# Thread 0:
# Thread 1:
# Thread 2:
# Thread 3:
```

图 20

请将 CPU 空载状态下的统计结果 no_load_output 文件拷贝至 Ubuntu，存放在 get_histogram.sh 脚本文件同一目录下，拷贝 no_load_output 文件并命名为 output。执行如下命令，get_histogram.sh 脚本文件添加可执行权限，运行脚本生成直方图文件 plot.png，请将其拷贝至 Windows 下并打开。

Host# chmod a+x get_histogram.sh

Host# ./get_histogram.sh

```
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$ ls
cyclictest  get_histogram.sh  no_load_output
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$ cp no_load_output output
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$ chmod a+x get_histogram.sh
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$ ls
cyclictest  get_histogram.sh  no_load_output  output
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$ ./get_histogram.sh
max latency: 190
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$ ls
cyclictest        histogram   histogram2  histogram4      output   plot.png
get_histogram.sh  histogram1  histogram3  no_load_output  plotcmd
tronlong@tronlong-virtual-machine:~/Cyclictest/bin$
```

图 21

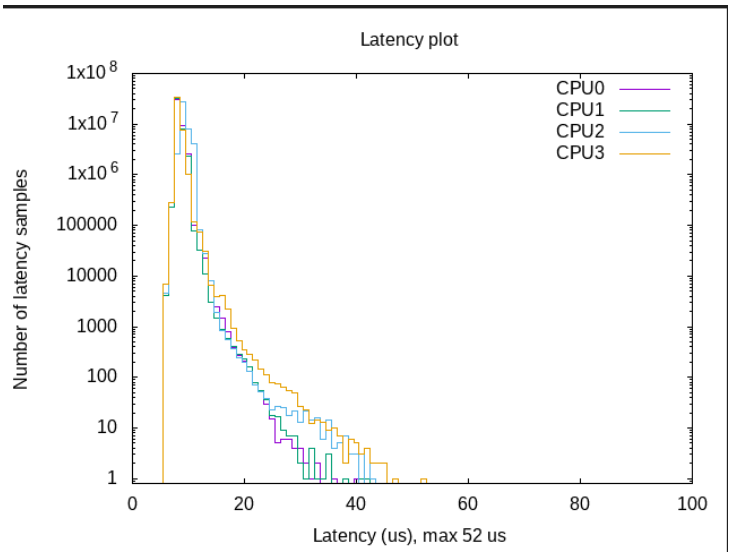

图 22

根据测试结果 output 文件数据以及结合直方图，可得主要数据如下表。本次测试中，CPU3 核心 Max Latencies 值最大，为 52us，CPU0 核心的 Max Latencies 值最小，为 40us。

备注：测试数据与实际测试环境有关，仅供参考。

表 2

CPU 核心 / Latencies	CPU0	CPU1	CPU2	CPU3
Min Latencies	6	6	6	6
Avg Latencies	8	8	9	8
Max Latencies	40	42	43	52

（2） CPU 满负荷状态

参考如上方法，分析 CPU 满负荷状态下的统计结果如下所示。本次测试中，CPU0 核心 Max Latencies 值最大，为 154us，CPU3 核心的 Max Latencies 值最小，为 94us。

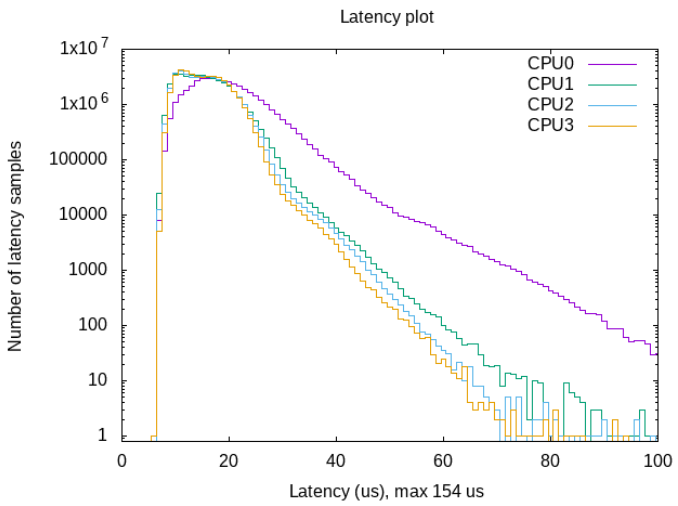

图 23

表 3

CPU / Latencies	CPU0	CPU1	CPU2	CPU3
Min Latencies	6	6	7	6
Avg Latencies	19	15	15	15
Max Latencies	154	103	108	94

（3） 隔离 CPU 核心状态

参考如上方法，分析隔离 CPU 核心状态下的统计结果如下所示。本次测试中，CPU0 核心 Max Latencies 值最大，为 190us，隔离 CPU3 核心的 Max Latencies 值最小，为 85us。

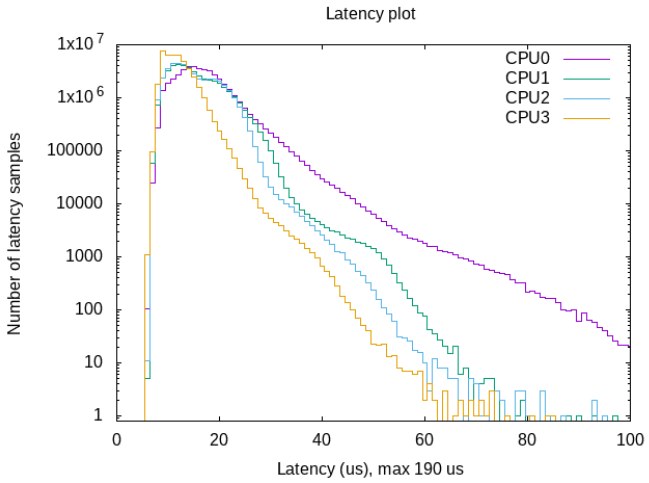

图 24

表 4

CPU / Latencies	CPU0	CPU1	CPU2	CPU3
Min Latencies	6	6	6	6
Avg Latencies	16	15	14	11
Max Latencies	190	113	101	85

根据 CPU 满负荷、隔离 CPU 核心状态的测试结果可知：当程序指定至隔离的 CPU3 核心上运行时，Linux 系统延迟最低，可有效提高系统实时性。故推荐对实时性要求较高的程序（功能）指定至隔离的 CPU 核心运行。

2 rt_gpio_ctrl 案例

2.1 案例说明

通过创建一个基本的实时线程，在线程内触发 LED 的电平翻转，同时程序统计实时线程的调度延时，并通过示波器测出 LED 电平两次翻转的时间间隔。由于程序默认以最高优先级运行，为避免 CPU 资源被程序完全占用导致系统被挂起，因此在程序中增加 100us 的延时。程序原理大致如下：

（1） 在 Linux-RT 内核上创建、使用实时线程。

（2） 实时线程中，计算出触发 LED 电平翻转的系统调度延时。

2.2 案例测试

本小节测试以隔离 CPU3 核心为例，通过降低系统上所运行的其他进程对隔离 CPU3 产生的延迟影响，确保 CPU3 进程的正常运行，进而评估 Linux-RT 内核的系统实时性。

评估板上电启动后，在 U-Boot 倒计时结束之前按下空格键进入 U-Boot 命令行模式，执行如下命令，修改环境变量，隔离 CPU3 核心。

U-Boot# setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} cpuidle.off=1 isolcpus=3"

U-Boot# saveenv

U-Boot# reset

```
u-boot=>
u-boot=> setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} cpuidle.off=1 isolcp
us=3"
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 25

```
root@IMX8MP-Tronlong:~# cat /proc/cmdline
console=ttymxc1,115200 root=/dev/mmcblk1p2 rootwait rw cpuidle.off=1 isolcpus=3
root@IMX8MP-Tronlong:~#
```

图 26

将案例 bin 目录下的可执行文件拷贝至评估板文件系统，并执行如下命令运行测试程序，再按"Ctrl + C"退出测试，串口终端将打印程序统计的延迟数据，如下图所示。

Target# taskset -c 3 ./rt_gpio_ctrl

```
root@IMX8MP-Tronlong:~# taskset -c 3 ./rt_gpio_ctrl
version : 1.0

This program outputs a square wave
with a pulse width of 100 us and a duty cycle of 50%.

The waveform can be obtained by measuring
the corresponding output pin with an oscilloscope.

'ctrl + c' to exit and get latency analyse results.
^C
Cycles : 2198542
Latency results : [Min] 12 us [Max] 30 us [Avg] 14 us
root@IMX8MP-Tronlong:~#
```

图 27

同时使用示波器捕捉 LED 两次电平翻转之间的间隔，即可得到线程调度的延迟，测试点为 R42 电阻一端。

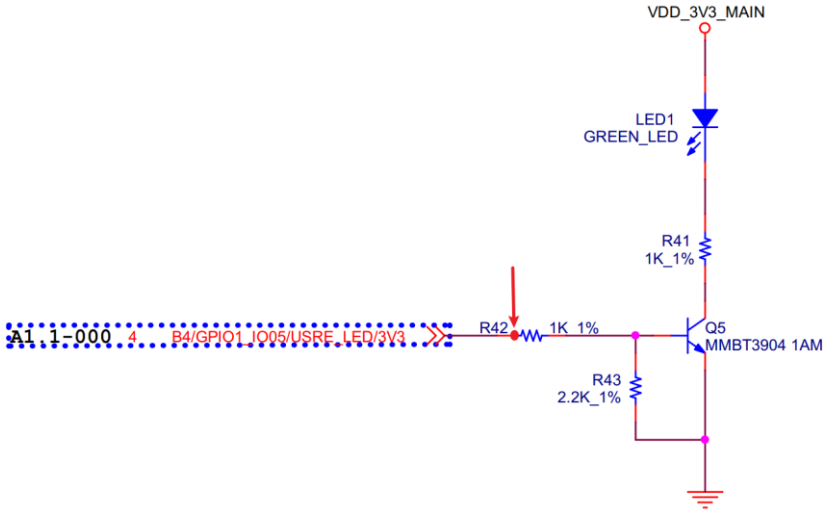

图 28

本次测得电平翻转周期为Δx = 115us，如下图所示。由于程序中默认增加了 100us 的时间延时，因此实际延时应为：115us - 100us = 15us，与程序统计打印的 Latency results 平均值相近。

图 29

2.3 案例编译

将产品资料“4-软件资料\Demo\rt-linux-demos\rt_gpio_ctrl\”案例源码拷贝至 Ubuntu。进入案例源码目录，执行如下命令，编译案例生成可执行文件。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# make

```
tronlong@tronlong-virtual-machine:~/rt_gpio_ctrl/src$ ls
makefile  rt_gpio_ctrl.c
tronlong@tronlong-virtual-machine:~/rt_gpio_ctrl/src$ source /home/tronlong/SDK/environ
ment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/rt_gpio_ctrl/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORT
IFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlon
g/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall
 -O3     -c -o rt_gpio_ctrl.o rt_gpio_ctrl.c
aarch64-poky-linux-g++  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORT
IFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlon
g/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -o rt_gp
io_ctrl rt_gpio_ctrl.o -Wl,-O1 -Wl,--hash-style=gnu -Wl,--as-needed  -Wl,-z,relro,-z,no
w -lpthread
tronlong@tronlong-virtual-machine:~/rt_gpio_ctrl/src$ ls
makefile  rt_gpio_ctrl  rt_gpio_ctrl.c  rt_gpio_ctrl.o
tronlong@tronlong-virtual-machine:~/rt_gpio_ctrl/src$
```

图 30

2.4 关键代码说明

（1） 创建实时任务，具体操作包括内存锁定、线程堆栈内存设置、调度策略和优先级配置等。

```c
    /* Lock memory */
    if (mlockall(MCL_CURRENT|MCL_FUTURE) == -1) {
        printf("mlockall failed: %m\n");
        exit(-2);
    }

    /* Initialize pthread attributes (default values) */
    if (pthread_attr_init(&attr) != 0) {
        printf("init pthread attributes failed\n");
        return -1;
    }

    /* Set a specific stack size  */
    if (pthread_attr_setstacksize(&attr, PTHREAD_STACK_MIN) != 0) {
        printf("pthread setstacksize failed\n");
        return -1;
    }

    /* Set scheduler policy and priority of pthread */
    if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0) {
        printf("pthread setschedpolicy failed\n");
        return -1;
    }

    param.sched_priority = 80;
    if (pthread_attr_setschedparam(&attr, &param) != 0) {
        printf("pthread setschedparam failed\n");
        return -1;
    }

    /* Use scheduling parameters of attr */
    if (pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED) != 0) {
        printf("pthread setinheritsched failed\n");
        return -1;
    }

    /* Create a pthread with specified attributes */
    if (pthread_create(&thread, &attr, thread_func, (void *)&rt_thread_params) != 0) {
        printf("create pthread failed\n");
        return -1;
    }

    /* Join the thread and wait until it is done */
    if (pthread_join(thread, NULL) != 0)
        printf("join pthread failed: %m\n");

    /* Unock memory */
    munlockall();

    return 0;
}
```

图 31

（2） 在线程中打开 LED 文件节点，并对 LED 状态进行翻转。

```c
    /* Open the LED node */
    int fd = open(LED, O_WRONLY);
    if (fd < 0) {
        printf("LED open failed.\n");
        return NULL;
    }

    bool flag = false;
    while (! g_quit) {
        clock_gettime(CLOCK_REALTIME, &t_info->t1);
        clock_nanosleep(CLOCK_MONOTONIC, 0, &t_info->interval, NULL);

        if (flag) {
            /* turn off LED */
            if (write(fd, "0\n", 2) < 0) {
                printf("Fail to turn off led.\n");
            }
        } else {
            /* turn on LED */
            if (write(fd, "1\n", 2) < 0) {
                printf("Fail to turn on led.\n");
            }
        }

        clock_gettime(CLOCK_REALTIME, &t_info->t2);

        /* analyse the latecy to get error range */
        analyse_latency(l_info, t_info, ++cycle);

        flag = !flag;
    }
```

图 32

（3） 统计调度时间延时情况。

```c
void analyse_latency(struct LatencyInfo *l_info,
                     struct TimeInfo *t_info,
                     long long cycle) {
    /* Calculating time difference */
    t_info->latency.tv_sec = t_info->t2.tv_sec -
                             t_info->t1.tv_sec - t_info->interval.tv_sec;

    t_info->latency.tv_nsec = t_info->t2.tv_nsec -
                              t_info->t1.tv_nsec - t_info->interval.tv_nsec;

    if (t_info->latency.tv_nsec < 0L) {
        t_info->latency.tv_sec --;
        t_info->latency.tv_nsec += 1000000000L;
    }

    /**
     * Conversion time to microseconds and
     * Calculating the maximum、 minimum and average of latency
     **/
    l_info->act_us = t_info->latency.tv_sec * 1000000 +
                     t_info->latency.tv_nsec / 1000;

    l_info->min_us  = l_info->act_us < l_info->min_us ?
                      l_info->act_us : l_info->min_us;

    l_info->max_us  = l_info->act_us > l_info->max_us ?
                      l_info->act_us : l_info->max_us;

    l_info->sum_us += l_info->act_us;
    l_info->avg_us  = l_info->sum_us / cycle;
    return;
}
```

图 33

3 rt_input 案例

3.1 案例说明

通过创建一个基本的实时线程，在线程内打开 input 设备，并对按键事件进行监听，然后触发 LED 的电平翻转，再通过示波器测量按键触发到 LED 电平翻转期间的实际耗时。程序原理大致如下：

（1） 在 Linux-RT 内核上创建、使用实时线程。

（2） 实时线程中对打开的 input 设备节点进行按键事件监听，通过判断监听得到的按键事件来触发 LED 的电平翻转。

3.2 案例测试

本小节测试以隔离 CPU3 核心为例，通过降低系统上所运行的其他进程对隔离 CPU3 产生的延迟影响，确保 CPU3 进程的正常运行，进而评估 Linux-RT 内核的系统实时性。

评估板上电启动后，在 U-Boot 倒计时结束之前按下空格键进入 U-Boot 命令行模式，执行如下命令，修改环境变量，隔离 CPU3 核心。

U-Boot# setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} cpuidle.off=1 isolcpus=3"

U-Boot# saveenv

U-Boot# reset

```
u-boot=>
u-boot=> setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} cpuidle.off=1 isolcp
us=3"
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 34

```
root@IMX8MP-Tronlong:~# cat /proc/cmdline
console=ttymxc1,115200 root=/dev/mmcblk1p2 rootwait rw cpuidle.off=1 isolcpus=3
root@IMX8MP-Tronlong:~#
```

图 35

由于系统默认打开 LED1，请执行如下命令关闭 LED1 避免影响后续测试。

Target# echo 0 > /sys/class/leds/user-led0/brightness

```
root@IMX8MP-Tronlong:~# echo 0 > /sys/class/leds/user-led0/brightness
root@IMX8MP-Tronlong:~#
```

图 36

将案例 bin 目录下的可执行文件拷贝至评估板文件系统，并执行如下命令运行测试程序，程序运行后按下 USER(KEY3)用户输入按键点亮 LED，松开按键后 LED 熄灭，再按"Ctrl + C"退出测试程序。

Target# taskset -c 3 ./rt_input /dev/input/event0

```
root@IMX8MP-Tronlong:~# taskset -c 3 ./rt_input /dev/input/event0
version : 1.0

Usage : rt_input [input device]  -- default : /dev/input/event0

e.g. : rt_input /dev/input/event0

This program function is gpio input trigger led output.

The waveform can be obtained by measuring
        the corresponding output pin with an oscilloscope.

'ctrl + c' to exit.
```

图 37

分别使用示波器探头 1 测量 LED 电路 R42 电阻一端，使用示波器探头 2 测量按键 USER KEY3 电阻 R45 引脚 1。

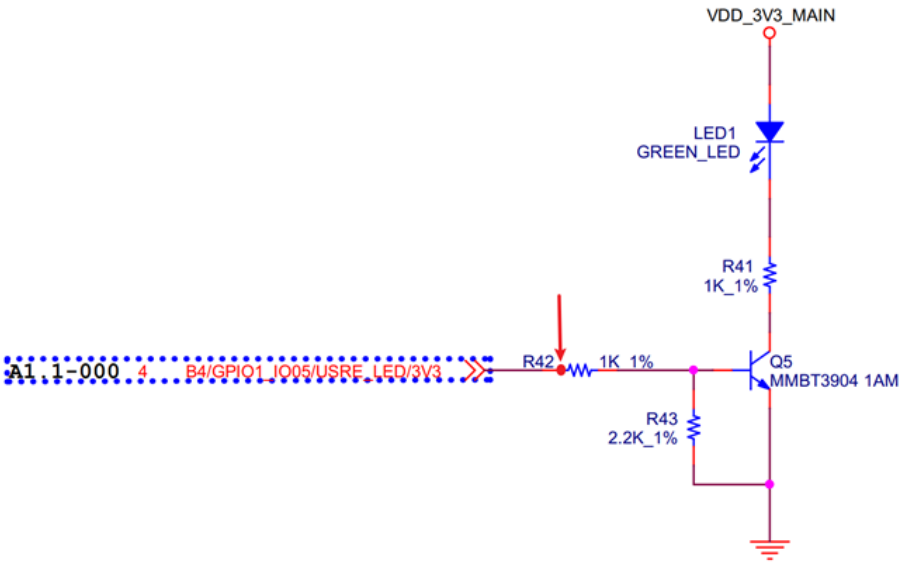

图 38

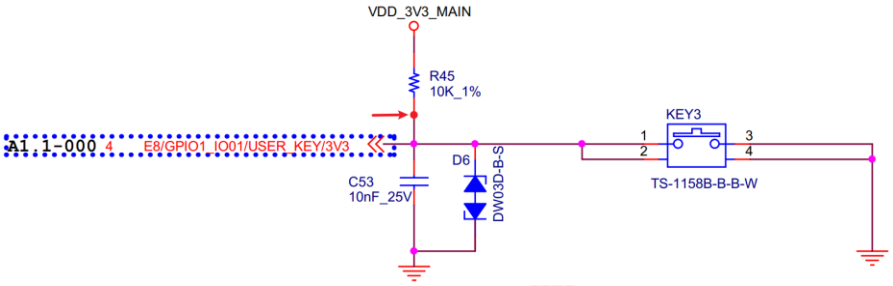

图 39

从按键下降沿触发的开始（下图蓝线）到 LED 上升沿触发的完成（下图黄线）的时间间隔，即为系统实时捕获按键输入时间并响应触发 LED 电平翻转的时间Δx，从图中可看到Δx = 88us。

图 40

备注：在硬件特性上，由于按键电压由低电平上拉到高电平比较缓慢，因此本次测试实时事件的输入采用下降沿触发方式。

3.3 案例编译

将产品资料“4-软件资料\Demo\rt-linux- demos\rt_input\”案例源码拷贝至 Ubuntu。进入案例源码目录，执行如下命令，编译案例生成可执行文件。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# make

```
tronlong@tronlong-virtual-machine:~/rt_input/src$ ls
makefile  rt_input.c
tronlong@tronlong-virtual-machine:~/rt_input/src$ source /home/tronlong/SDK/environment
-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/rt_input/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORT
IFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlon
g/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall
 -O3     -c -o rt_input.o rt_input.c
aarch64-poky-linux-g++  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORT
IFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlon
g/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -o rt_in
put rt_input.o -Wl,-O1 -Wl,--hash-style=gnu -Wl,--as-needed  -Wl,-z,relro,-z,now -lpthr
ead -lrt
tronlong@tronlong-virtual-machine:~/rt_input/src$ ls
makefile  rt_input  rt_input.c  rt_input.o
tronlong@tronlong-virtual-machine:~/rt_input/src$
```

图 41

3.4 关键代码说明

（1） 创建实时任务，具体操作包括内存锁定、线程的堆栈内存设置、调度策略和优先级配置等。

```c
    /* Lock memory */
    if (mlockall(MCL_CURRENT|MCL_FUTURE) == -1) {
        printf("mlockall failed: %m\n");
        exit(-2);
    }

    /* Initialize pthread attributes (default values) */
    if (pthread_attr_init(&attr) != 0) {
        printf("init pthread attributes failed\n");
        return -1;
    }

    /* Set a specific stack size  */
    if (pthread_attr_setstacksize(&attr, PTHREAD_STACK_MIN) != 0) {
        printf("pthread setstacksize failed\n");
        return -1;
    }

    /* Set scheduler policy and priority of pthread */
    if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0) {
        printf("pthread setschedpolicy failed\n");
        return -1;
    }

    param.sched_priority = 80;
    if (pthread_attr_setschedparam(&attr, &param) != 0) {
        printf("pthread setschedparam failed\n");
        return -1;
    }

    /* Use scheduling parameters of attr */
    if (pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED) != 0) {
        printf("pthread setinheritsched failed\n");
        return -1;
    }

    /* Create a pthread with specified attributes */
    if (pthread_create(&thread, &attr, thread_func, (void *)&rt_thread_params) != 0) {
        printf("create pthread failed\n");
        return -1;
    }

    /* Join the thread and wait until it is done */
    if (pthread_join(thread, NULL) != 0)
        printf("join pthread failed: %m\n");

    /* Unock memory */
    munlockall();

    return 0;
}
```

图 42

（2） 在线程中打开 input 设备节点并监听按键事件，同时触发 LED 电平的翻转。

```
71      while (! g_quit) {
72          fd_set input;
73          FD_ZERO(&input);
74          FD_SET(fd[0], &input);
75
76          timeout.tv_sec = 0;
77          timeout.tv_usec = 300000;
78
79          int ret = select(fd[0] + 1, &input, NULL, NULL, &timeout);
80          if (ret == 0)
81              continue;
82
83          if (ret < 0) {
84              printf("%s", strerror(errno));
85              goto Release;
86          }
87
88          struct input_event buf;
89          if (read(fd[0], &buf, sizeof(struct input_event)) < 0) {
90              printf("%s", strerror(errno));
91              goto Release;
92          }
93
94          switch (buf.code) {
95          case KEY_PROG1:
96              if (buf.value == 1)     // 1: pressed; 0: released
97                  if (write(fd[1], "1\n", 2) < 0)
98                      printf("Fail to turn on led.\n");
99              if (buf.value == 0)
100                 if (write(fd[1], "0\n", 2) < 0)
101                     printf("Fail to turn off led.\n");
102
103             break;
104         case KEY_PROG2:
105             if (buf.value == 1)     // 1: pressed; 0: released
106                 if (write(fd[1], "1\n", 2) < 0)
107                     printf("Fail to turn on led.\n");
108             if (buf.value == 0)
109                 if (write(fd[1], "0\n", 2) < 0)
110                     printf("Fail to turn off led.\n");
111             break;
112         case KEY_PROG3:
113             if (buf.value == 1)     // 1: pressed; 0: released
114                 if (write(fd[1], "1\n", 2) < 0)
115                     printf("Fail to turn on led.\n");
116             if (buf.value == 0)
117                 if (write(fd[1], "0\n", 2) < 0)
118                     printf("Fail to turn off led.\n");
119
120             break;
121         default:
122             break;
123         }
124     }
```

图 43

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com